Spring Boot Actuator

Introduction:
Spring Boot Actuator is a **built-in module** that provides **production-ready features** to help you **monitor and manage** your Spring Boot application through a set of **RESTful endpoints**.

Why are we using Actuator?
We use Actuator to:

- **Monitor application health**
- **Track runtime metrics** (CPU, memory, threads, etc.)
- **Expose environment details**
- **View request mappings, bean details, log levels**
- **Integrate with monitoring tools** like Prometheus, Grafana

It simplifies diagnostics and enables real-time visibility into the application.

Why is it required / why was it introduced?
Before Actuator:

- Developers had to write **custom monitoring and health-check logic**.
- There was **limited visibility** into internal states during runtime.
- No native support for **cloud-native monitoring and DevOps tools**.

Actuator solves this by offering **out-of-the-box endpoints** for common operational needs, making applications **easier to maintain, debug, and scale**.

🚫 **What if Actuator didn't exist**?
Without Actuator:

- Developers had to **build custom health checks and metrics manually**.
- Difficult to monitor **runtime behavior** (e.g., memory, thread usage).
- **No visibility** into Spring beans, mappings, or configs.
- Harder to integrate with **cloud platforms** or monitoring tools.
- **No centralized access** to critical application info during production.

📌 **In Summary**:

- Actuator gives deep insight into **Spring Boot internals**.
- It acts as a **data source** for advanced tools like **Grafana or Dynatrace**.
- Using both together gives you the best of **application-level** and **infrastructure-level** monitoring.

Akash Kumar

📖 Common Actuator Endpoints & Sample Responses

Endpoint	Description	Sample Response
`/actuator/health`	App health check / Shows application health	`{ "status": "UP" }`
`/actuator/info`	App info (e.g., version) / Displays arbitrary application info	`{ "app": { "version": "1.0.0" } }`
`/actuator/metrics`	All metrics available / Shows metrics like memory, threads, GC, etc	`{ "names": ["jvm.memory.used", "http.server.requests", ...] }`
`/actuator/metrics/jvm.memory.used`	Specific metric value	`{ "measurements": [{ "value": 123456 }], "availableTags": [...] }`
`/actuator/env`	All environment properties / Shows environment properties	`{ "activeProfiles": [], "propertySources": [...] }`
`/actuator/beans`	List all beans / Lists all Spring Beans	`{ "contexts": { "application": { "beans": [...] }}}`
`/actuator/mappings`	All controller endpoints	`{ "/api/users": "UserController" }`
`/actuator/loggers`	Logging config & levels / Manage and view log levels	`{ "levels": ["OFF", "ERROR", "WARN", "INFO", "DEBUG", "TRACE"] }`

Akash Kumar

✅ If other monitoring tools are there (like Grafana, Dynatrace), Why do we need an Actuator?

Purpose	Spring Boot Actuator	External Tools (e.g., Grafana, Dynatrace)
Provides **internal Spring Boot metrics** (beans, mappings, env)	✅	❌
Easily enables **custom application health checks**	✅	❌
Offers **out-of-the-box Spring context visibility**	✅	❌
Collects low-level metrics	⚠️ (basic)	✅ (advanced)
Visual dashboards & alerting	❌	✅
Requires complex setup	❌	✅ (often complex)

✅Can Actuator be configured with other monitoring tools?

Yes, Spring Boot Actuator integrates seamlessly with **external monitoring systems** using **Micrometer** (built into Actuator).
Popular integrations include:

- **Prometheus** – scrape metrics from `/actuator/prometheus`
- **Grafana** – visualize metrics pulled from Prometheus
- **New Relic**, **Datadog**, **InfluxDB**, **Wavefront**, etc.

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

✅How can we implement Actuator in a Spring Boot application?

Step-by-Step

Step - 1 - Add Actuator Dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Step 2: Configure Endpoints in application.properties

```
management.endpoints.web.exposure.include=health,info,metrics
management.endpoints.web.base-path=/actuator
```

OR

```
# Expose specific endpoints (comma-separated)
management.endpoints.web.exposure.include=health,info,metrics

# Or expose all endpoints (not recommended for production)
management.endpoints.web.exposure.include=*

# Set custom context path for actuator endpoints
management.endpoints.web.base-path=/actuator

# Customize port for actuator endpoints (optional)
management.server.port=8081
```

Step 3: **Custom Endpoint (Optional Code Example)**

```
@RestController
public class CustomEndpoint {

    @GetMapping("/custom/health")
    public String customHealthCheck() {
        return "Service is running";
    }
}
```

Step 4: Run the Application

Access endpoints like:

- `http://localhost:8080/actuator/health`
- `http://localhost:8080/actuator/metrics`

📌 Purpose of Spring Boot Actuator

Spring Boot Actuator provides **production-ready monitoring and management endpoints** to inspect and control your application **without changing business logic**.
It helps:

- Monitor **health, metrics, environment**, and logs.
- Detects **performance bottlenecks** or failures quickly.
- Support integration with **DevOps tools** like Prometheus, Grafana, etc.
- Enable **runtime control** (like changing log levels) safely.

📌 Benefits of Spring Boot Actuator

1. **Production-Ready Monitoring**
   Provides built-in endpoints for health, metrics, logging, environment, etc., with zero custom code.

2. **Easy Integration**
   Works well with monitoring tools like **Prometheus, Grafana, ELK Stack**, etc.

3. **Real-Time Insights**
   Helps track JVM usage, request metrics, and bean health instantly.

4. **Customizable and Extensible**
   You can add **custom health indicators**, **metrics**, and even your own endpoints.

5. **DevOps & Cloud Friendly**
   Supports dynamic log level changes, health checks for load balancers, and Kubernetes probes (/health).

6. **Reduces Debugging Effort**
   Exposes detailed info about beans, environment, mappings, and thread dumps during runtime.

In Summary:

- Actuator gives deep insight into **Spring Boot internals**.
- It acts as a **data source** for advanced tools like **Grafana or Dynatrace**.
- Using both together gives you the best of **application-level** and **infrastructure-level** monitoring.

Akash Kumar

⚠️ Disadvantages of Spring Boot Actuator

1. **Security Risk if Not Configured Properly**
   Exposing sensitive endpoints like `/env`, `/beans`, or `/configprops` without authentication can lead to vulnerabilities.
2. **Performance Overhead**
   Collecting detailed metrics can add **slight runtime overhead**, especially under high traffic if not fine-tuned.
3. **Verbose Output**
   Some endpoints return large JSON responses, which may not be suitable in minimal or resource-constrained systems.
4. **Limited Custom Metrics by Default**
   While basic metrics are provided, more complex business-specific metrics require manual implementation.
5. **Not All Endpoints are Enabled by Default**
   Requires configuration to explicitly enable or expose certain endpoints.

🚫 What if Actuator didn't exist?

Without Actuator:

- Developers had to **build custom health checks and metrics manually**.
- Difficult to monitor **runtime behavior** (e.g., memory, thread usage).
- **No visibility** into Spring beans, mappings, or configs.
- Harder to integrate with **cloud platforms** or monitoring tools.
- **No centralized access** to critical application info during production.

Thank you

Akash Kumar